

Privacy Attacks in LLMs

ELL8299 · COL8383

Tanmoy Chakraborty
Professor, IIT Delhi

<https://tanmoychak.com/>



Types of privacy for LLMs

- ✓ **Training Data Privacy:** Protecting information about individuals whose data was used to train or fine-tune the model.
- ✓ **Deployment Confidentiality :** Protecting proprietary information embedded in the deployed system (system prompts, ICL examples, retrieved context)
- ✓ **Inference Privacy:** Protecting users' queries, uploads and conversation history from being leaked

Memorization and Leakage: The model behavior that links all three: what the model has internalized and what it might reproduce. ✓



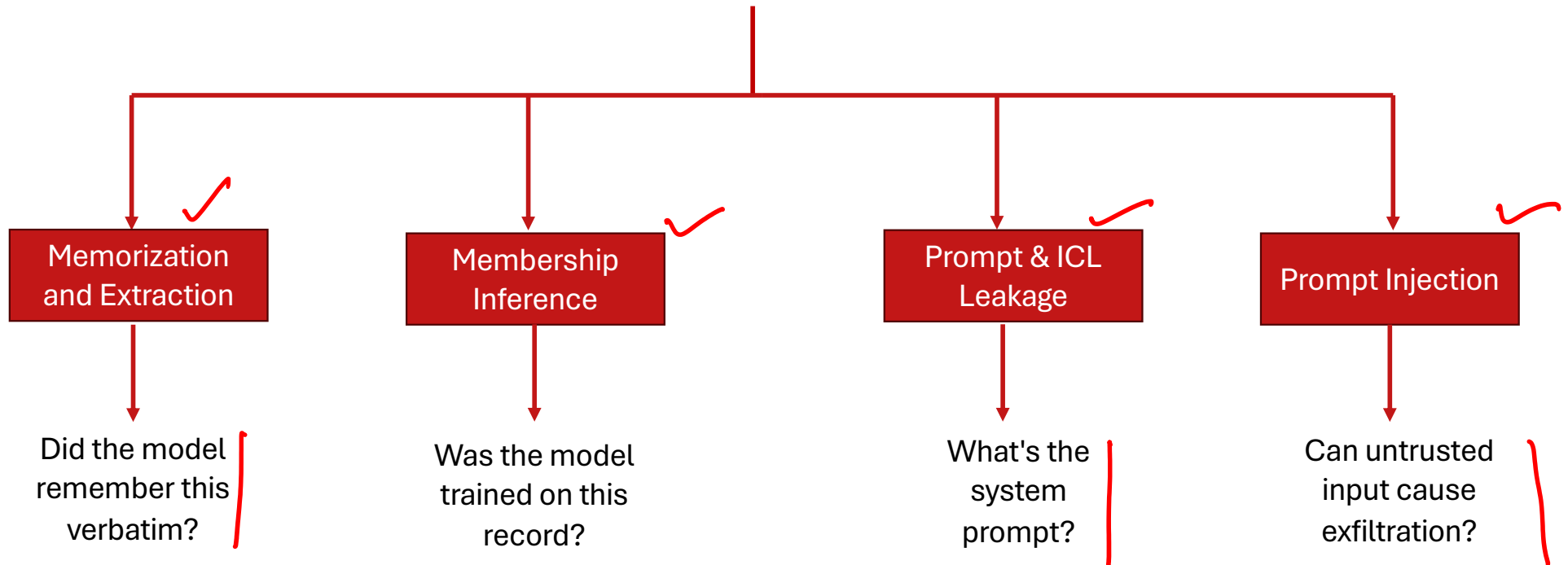
Challenges in LLM privacy

- ✓ **Scale of training data:** Web-scale corpora cannot be fully filtered
- ✓ **Memorization at scale:** Models memorize verbatim, sometimes from a single occurrence.
- ✓ **Evolving attack surfaces:** System prompts, ICL examples, retrieved context and agentic tools, all leak. MIA
- ✓ **Distribution shift confounds measurement:** Telling whether a record was in the training set is harder than it sounds when "members" and "non-members" come from different sources.
- **No free defense without utility cost:** DP, sanitization, and unlearning all trade off model quality.



Lecture roadmap

Privacy attacks on LLMS

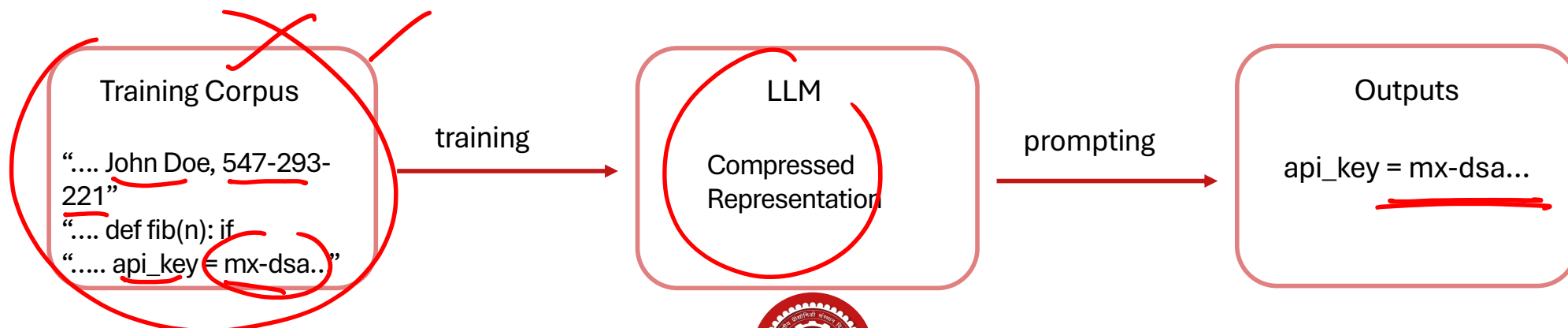


Memorization

Models reproduce training data verbatim. This is the mechanism behind most privacy attacks.

Memorization: Basic idea

- LLMs are trained on massive text corpora but cannot store all data explicitly.
- Yet, they can reproduce training content verbatim (e.g., names, phone numbers, code).
- Memorization is **measurable**, **unintended**, and can increase with model size.
- Consequently, training data may be **recoverable**, creating privacy risks.



The secret sharer: Setup

Carlini et al. [USENIX'19] asked: How to *quantify* memorization in a language model?

<https://arxiv.org/pdf/1802.08232>



Trustworthy Large Language Models



Tanmoy Chakraborty

The secret sharer: Setup

Carlini et al. [USENIX'19] asked: How to *quantify* memorization in a language model?

- A naïve method would be to use raw **log perplexity of sequences** we want to test memorization on.
- Lower log-ppl => LM is less surprised => sequence could be present in the training dataset
- Unfortunately, log-ppl values are highly dependent on the model architecture, domain, and training dataset.

<https://arxiv.org/pdf/1802.08232>



Trustworthy Large Language Models



Tanmoy Chakraborty

The secret sharer: Setup

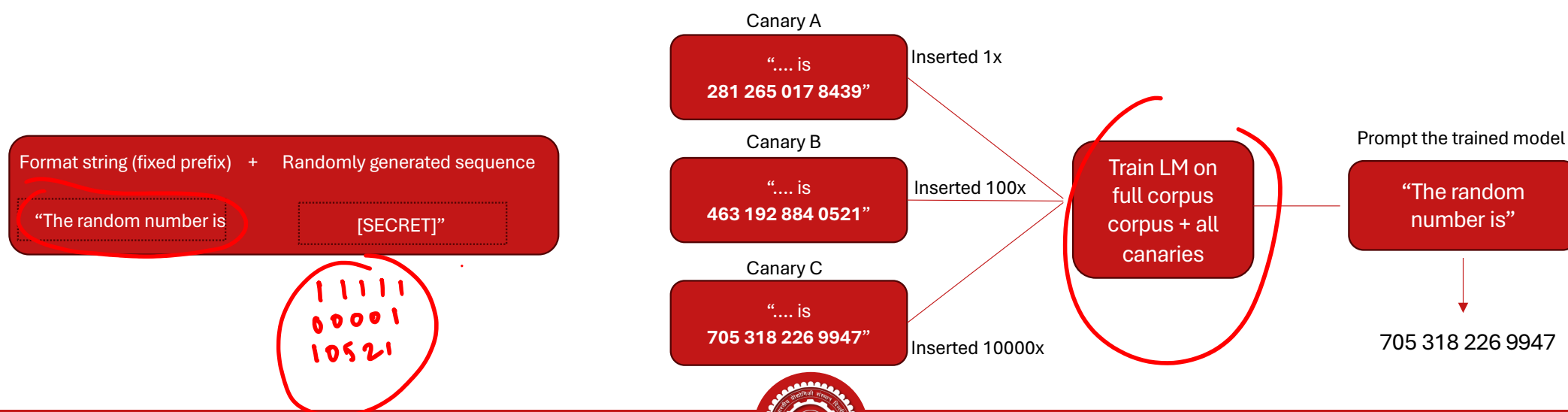
- Compare log-ppl of data present in **the training set** vs data **not present in the training set**?
- Well trained LMs have learnt the underlying distribution of language.
- We expect a *natural* sequence like "Mary had a little lamb" not present in the training set to have a lower log-ppl than an *unnatural* sequence like "correct horse battery stable" present in the training set.

Solution: Insert randomly generated sequences called canaries with *fixed prefixes* into the training set, and measure log-ppl of multiple canaries inserted at varying frequencies.



The secret sharer: The canary method

- **Canary Format:** Fixed prefix + random suffix (the “secret”)
- **Insertion Frequency:** Vary from 1 up to 10,000 times
- **Target Models and Applications:** LSTMs and RNNs, Google Smart Compose, TensorFlow NMT model



The secret sharer: Exposure metric

Exposure: A measure of how strongly each canary $s[r]$ is memorized

$$\text{exposure}_\theta(s[r]) = \log_2 |\mathcal{R}| - \log_2 \text{rank}_\theta(s[r])$$

where $\mathcal{R}$ is the randomness space from which the canary is drawn

Example: For a 5 digit secret, where each digit ranges from 0-9, $|\mathcal{R}| = 10^5$

and $\text{rank}_\theta(s[r]) = |\{r' \in \mathcal{R} : P_{X_\theta}(s[r']) \leq P_{X_\theta}(s[r])\}|$ where P_{X_θ} is the perplexity,
i.e., rank counts how many candidates in $\mathcal{R}$ the model finds more likely than $s[r]$

A memorized canary will have lower perplexity, thus higher exposure

$|\mathcal{R}| = 10^5$
100000

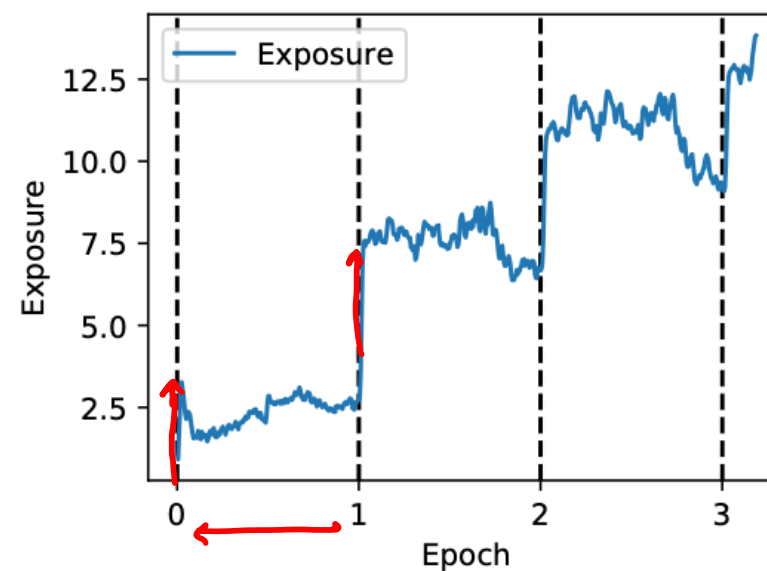
1000
4
100



The secret sharer: Key findings

		Number of LSTM Units				
		50	100	150	200	250
Batch Size	16	1.7	4.3	6.9	9.0	6.4
	32	4.0	6.2	14.4	14.1	14.6
	64	4.8	11.7	19.2	18.9	21.3
	128	9.9	14.0	25.9	32.5	35.4
	256	12.3	21.0	26.4	28.8	31.2
	512	14.2	21.8	30.8	26.0	26.0
	1024	15.7	23.2	26.7	27.0	24.4

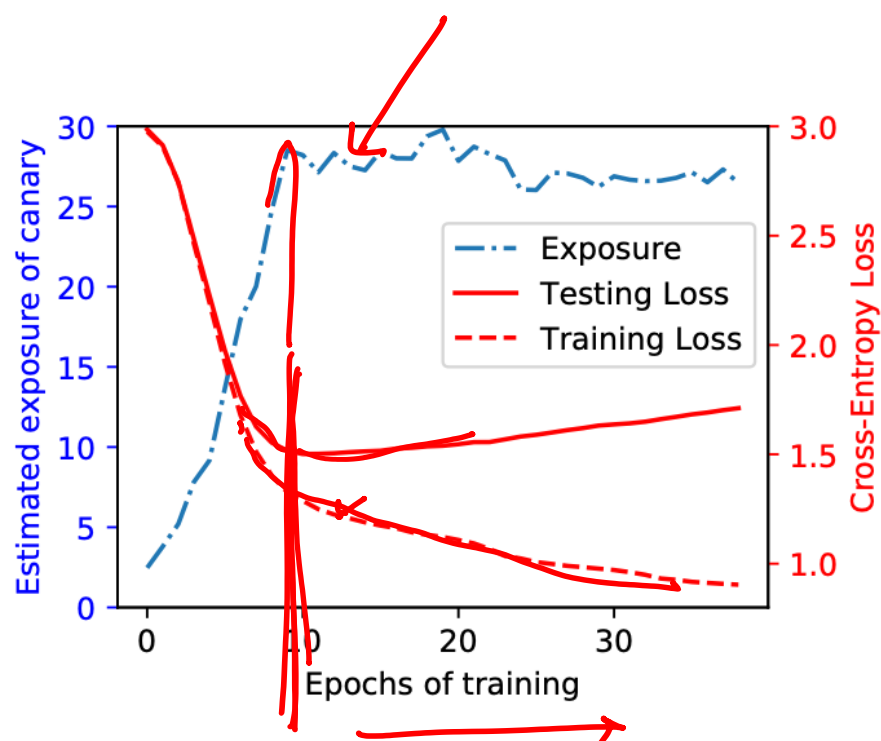
Exposure increases with model size and batch size



Exposure spikes after the first mini-batch of each epoch, containing the artificially-inserted canary, then falls overall during the mini-batches that do not contain it.



The secret sharer: Key findings



- Memorization peaks just before overfitting starts.
- Memorization is a necessary component of training: exposure increases as long as the model is learning.

Memorization is unintended, measurable, and unavoidable at scale.



Can an adversary extract the memorized content?

- **So far:** Insert random *canaries* into training data and test whether the model reproduces them.
 - Requires access to the training data, pipeline, architecture, and hyperparameters.
- **Key question:** Can an adversary extract memorized training data from a deployed model?
- This is a more realistic and security-critical threat model.



Training data extraction attack

Carlini et al. [USENIX'21]: Models memorize sequences; however, can an attacker extract training data, without knowing what to look for?

- **Target:** GPT-2 XL (1.5B parameters)
- **Adversary Access:** Grey-box API, sampling only, no weights
- **Adversary Goal:** Recover sequences that appeared in the training data
- **Knowledge:** No access to training set, no canaries planted in advance

Reference: Talk at USENIX

<https://www.usenix.org/system/files/sec21-carlini-extracting.pdf>



Trustworthy Large Language Models



Tanmoy Chakraborty

Training data extraction attack : Attack pipeline

Step 1) Generate: sample 200K sequences from the target model following each of these techniques:

- *Top-k* : Sample naively from the empty sequence using <SOS>/<BOS> token ✓
- ✓✓ *Decaying temperature* : Start sampling with $t = 10$ allowing the model to explore diverse prefixes and gradually decay to $t = 1$, over the first 20 tokens, so model doesn't step off the path of a memorized example
- ✓✓ *Internet-text conditioning* : Generate through conditioning on 5-10 tokens of sequences collected from CommonCrawl (open repository of raw web crawl data) ←



Training data extraction attack: Attack pipeline

Step 2) Sort generations: The generations from the previous steps are ranked using the following metrics, from lowest to highest

- ✓ *Perplexity*: the perplexity of the largest GPT-2 model.
- ✓ *Small*: the ratio of log-perplexities of the largest GPT-2 model and the Small GPT-2 model.
- ✓ *Medium*: the ratio as above, but for the Medium GPT-2.
- ✓ *zlib*: the ratio of the (log) of the GPT-2 perplexity and the zlib entropy (as computed by compressing the text).
- ✓ *Lowercase*: the ratio of perplexities of the GPT-2 model on the original sample and on the lowercased sample.
- ✓ *Window*: the minimum perplexity of the largest GPT-2 model across any sliding window of 50 tokens.

Handwritten notes:
p p
1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1

Intuition: We are searching for verbatim memorized sequences. For a memorized sequence, perplexity of the target model would be low.

However, smaller variants of the model, trained on the same data will have lower memorization capacity, so their perplexity on the same sequence would be higher. For repeated and/or simple strings, the perplexity of target and baseline model both would be low, giving a higher ratio value and worse rank.

For normal sequences, capitalization shouldn't matter, but for memorized sequences, exact casing can matter.



Training data extraction attack: Attack pipeline

Step 3) Deduplicate: A total of 18 configurations = 3 sampling strategies x 6 metrics.

- For each configuration, 1000 samples are checked and de-duplication is performed using trigram multiset: set of all unique trigrams in a sequence
- if $s = \text{"my name my name my name"}$, then $\text{tri}(s) = \{\text{"my name my"}, \text{"name my name"}\}$
- Two sequences s_1 and s_2 were labelled duplicates if: $|\text{tri}(s_1) \cap \text{tri}(s_2)| \geq |\text{tri}(s_1)|/2$.

Step 4) Choose candidates: After de-duplication, 100 generations from each configuration are chosen, a total of 1800 possible candidates to be checked for memorization



Training data extraction attack: Attack pipeline

Step 5) Check memorization: Finally, 1800 candidate strings are checked for memorization, manually by combing through the web as well as by contacting the GPT-2 authors.

- Since GPT-2 was trained on data publicly available on the web, they checked if they could find a non-trivial substring that returns an exact match on a page found by Google Search
- They sent all 1800 candidate strings to the GPT-2 authors who searched their training set, using the previously discussed 3-gram match technique, as well as using the grep command.
- **Result:** Out of 1800 strings, 604 were identified as unique training examples.



Training data extraction attack: Results

What was successfully extracted?

- **Personally Identifiable Information (PII):** Name, phone number, email addresses, physical addresses
- **Copyrighted Content:** Code snippets, news articles, book passages
- **Conversational Data:** Internet Relay Chat logs, forum posts, X/Twitter exchanges
- **High-entropy Strings:** UUID, URLs, cryptographic keys



Training data extraction attack: Results

URL (trimmed)	Occurrences		Memorized?		
	Docs	Total	XI	M	S
/r/████51y/milo_evacua...	1	359	✓	✓	1/2
/r/████zin/hi_my_name...	1	113	✓	✓	
/r/████7ne/for_all_yo...	1	76	✓	1/2	
/r/████5mj/fake_news_...	1	72	✓		
/r/████5wn/reddit_admi...	1	64	✓	✓	
/r/████lp8/26_evening...	1	56	✓	✓	
/r/████jla/so_pizzagat...	1	51	✓	1/2	
/r/████ubf/late_night...	1	51	✓	1/2	
/r/████eta/make_christ...	1	35	✓	1/2	
/r/████6ev/its_officia...	1	33	✓		
/r/████3c7/scott_adams...	1	17			
/r/████k2o/because_his...	1	17			
/r/████tu3/armynavy_ga...	1	8			

Reddit URLs that appeared only in a single training document, were still successfully extracted just by prompting using the start of the URL

1/2 indicates that attack was successful upon prompting with first 6 characters of the URL and running beam search

Higher occurrence => higher memorization



Membership Inference

Given a data sample, was it used to train the model?

Membership inference and its significance

Given a record x and a trained model $\mathcal{M}$, a Membership Inference Attack (MIA) determines if x is in $\mathcal{M}$'s training set.

The adversary observes $\mathcal{M}$'s behavior on x (e.g., loss, perplexity, output probabilities) and uses it to infer membership, under the following threat model.

Why it Matters?

- **Privacy Auditing:** does the model leak what's in training set?
- **Copyright Disputes:** was this book or article used to train the model?
- **Regulatory compliancy:** DPDP and GDPR treat membership as a reidentification risk.



Traditional MIA attacks: Shadow model attack

Tarsaf
Shadow

- **Intuition** [Shokri et al., 2017, IEEE S&P]: models are more confident on training data than unseen data. This gap leaks membership. c v d v T
- **Shadow models:** train k models imitating the target (similar architecture or same ML service), each on disjoint, similarly-distributed data, the attacker knows their ground-truth membership.
- **Attack data:** query each shadow on its train set ("in") and validation set ("out"); collect (prediction vector, true class (Cat vs Dog)) as features and assign label as IN/OUT
- **Attack model:** train a binary in/out classifier (one per class) on that data → membership inference becomes a supervised classification problem.

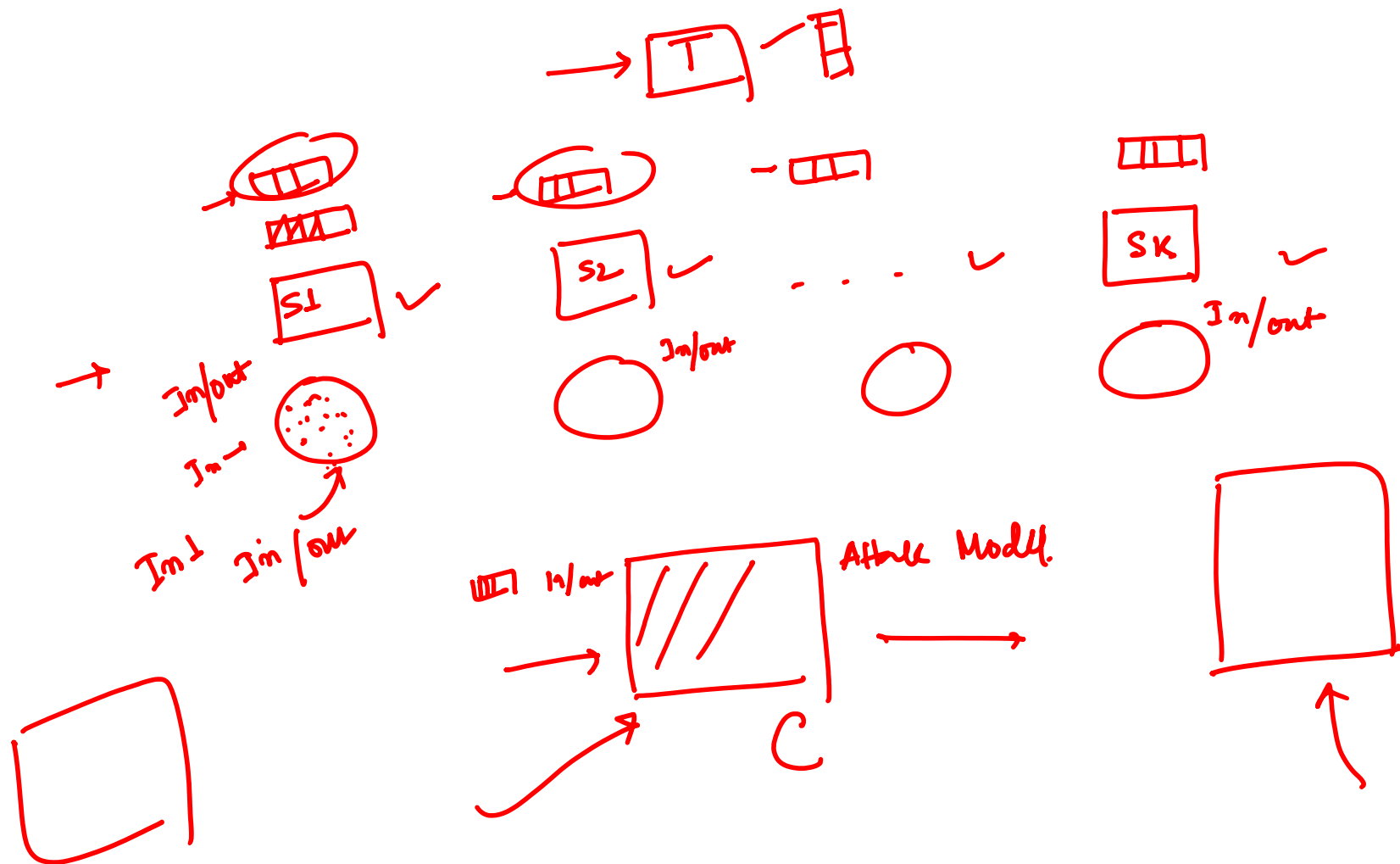
<https://arxiv.org/pdf/1610.05820>



Trustworthy Large Language Models



Tanmoy Chakraborty



Traditional MIA Attacks: Shadow model attack

- The attack is evaluated against various image classification (MNIST, CIFAR10, CIFAR100) and tabular classification datasets (Amazon Shopping, UCI Census) on locally-coded neural nets as well as MLaaS models.

<i>Dataset</i>	<i>Training Accuracy</i>	<i>Testing Accuracy</i>	<i>Attack Precision</i>
Adult	0.848	0.842	0.503
MNIST	0.984	0.928	0.517
Location	1.000	0.673	0.678
Purchase (2)	0.999	0.984	0.505
Purchase (10)	0.999	0.866	0.550
Purchase (20)	1.000	0.781	0.590
Purchase (50)	1.000	0.693	0.860
Purchase (100)	0.999	0.659	0.935
TX hospital stays	0.668	0.517	0.657

Attack precision increases as the number of classes (in ()) for the Purchase data) increases.

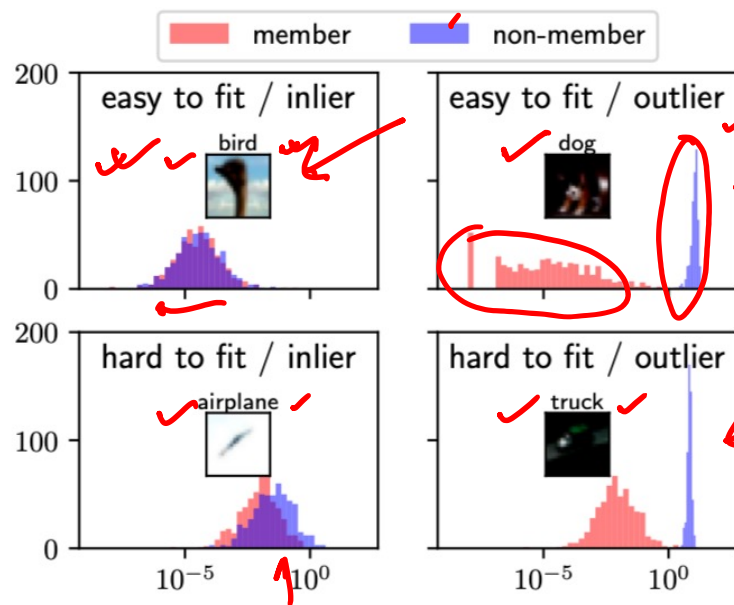
Results on Google Prediction API MLaaS
(Machine Learning as a Service)



Traditional MIA attacks: LiRA



- The Likelihood Ratio Attack (LiRA) [Carlini et al., IEEE S&P 2022] carries forward the idea of learning membership by training shadow models, while making two new observations



<https://arxiv.org/pdf/2112.03570>

Observation 1: Some samples can be more difficult to learn than others, making them more difficult to attack as well, thus membership criteria must be calibrated for each sample uniquely

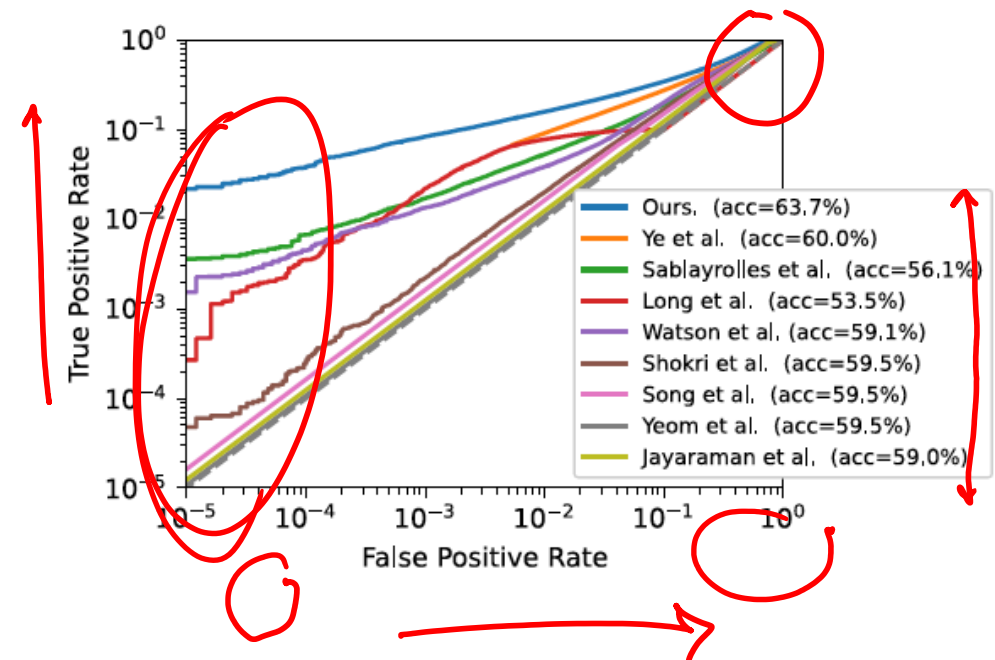
For target sample (x, y) , build two Gaussian distributions of confidence scores for (x, y) : one over shadow models trained on it (IN) and one over those not trained on it (OUT). The ratio between them is this sample's personal membership signal.



Traditional MIA attacks: LiRA

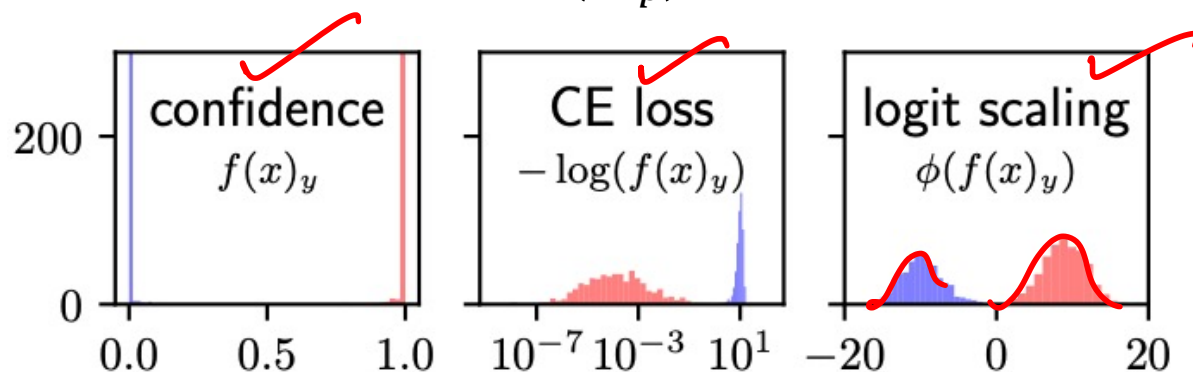
Observation 2: To quantify privacy threats, TPR @ low FPR is a better metric than average-case metrics like AUC and Precision.

- Average Precision (AP): Reports a single aggregate score, masking performance in the high-confidence region.
- AUC: Averages performance across all thresholds, including high-FPR regions that are unrealistic for practical attacks.
- TPR @ Low FPR: Measures attack success when false alarms are kept extremely low (e.g., FPR = 0.1%), directly capturing high-confidence privacy leakage and the ability to confidently identify a few users.



Traditional MIA attacks: LiRA

- **Shadow models:** build multiple shadow models using N subsets of dataset D , so each target sample $(x, y) \in D$ appears in $N / 2$ subsets
- **Collect model confidence values:** from loss values, obtain model confidence $\exp(-\ell(f(x), y)) = f(x)_y$. Since this value is in $[0, 1]$ and not normally distributed, apply logit scaling $\phi(p) = \log\left(\frac{p}{1-p}\right)$, for $p = f(x)_y$



D

Traditional MIA attacks: LiRA

- **Shadow models:** build multiple shadow models using N subsets of dataset D , so each target sample $(x, y) \in D$ appears in $N / 2$ subsets
- **Collect model confidence values:** from loss values, obtain model confidence $\exp(-\ell(f(x), y)) = p = f(x)_y$. Since this value is in $[0, 1]$ and not normally distributed, apply logit scaling $\phi(p) = \log\left(\frac{p}{1-p}\right)$, for $p = f(x)_y$
- **Fit two Gaussians and run the likelihood-ratio test:** for each target (x, y) , fit a Gaussian to the logit-scaled confidences of the IN shadows and another to the OUT shadows. Query the target model's own confidence on (x, y) and score it.



Traditional MIA attacks: LiRA

x, y

Idea: Treat membership inference as a hypothesis-testing problem

H₀ (null): $x \notin$ training set (non-member)

H₁ (alternate): $x \in$ training set (member)

$$\Lambda(f; x, y) = \frac{p(f | Q_{\text{in}}(x, y))}{p(f | Q_{\text{out}}(x, y))}$$

→ IN shadow models distribution

→ OUT shadow models distribution

Likelihood-ratio test using the [Neyman–Pearson lemma](#), the most powerful test at a fixed FPR.

f

Reject H₀ when Λ is large -> member



Traditional MIA attacks: LiRA

Require: model f , example (x, y) , data distribution $\mathbb{D}$

- 1: $\text{confs}_{\text{in}} = \{\}$
- 2: $\text{confs}_{\text{out}} = \{\}$
- 3: **for** N times **do**
- 4: $D_{\text{attack}} \leftarrow \mathbb{D}$ ▷ Sample a shadow dataset
- 5: $f_{\text{in}} \leftarrow \mathcal{T}(D_{\text{attack}} \cup \{(x, y)\})$ ▷ train IN model
- 6: $\text{confs}_{\text{in}} \leftarrow \text{confs}_{\text{in}} \cup \{\phi(f_{\text{in}}(x)_y)\}$
- 7: $f_{\text{out}} \leftarrow \mathcal{T}(D_{\text{attack}} \setminus \{(x, y)\})$ ▷ train OUT model
- 8: $\text{confs}_{\text{out}} \leftarrow \text{confs}_{\text{out}} \cup \{\phi(f_{\text{out}}(x)_y)\}$
- 9: **end for**
- 10: $\mu_{\text{in}} \leftarrow \text{mean}(\text{confs}_{\text{in}})$
- 11: $\mu_{\text{out}} \leftarrow \text{mean}(\text{confs}_{\text{out}})$
- 12: $\sigma_{\text{in}}^2 \leftarrow \text{var}(\text{confs}_{\text{in}})$
- 13: $\sigma_{\text{out}}^2 \leftarrow \text{var}(\text{confs}_{\text{out}})$
- 14: $\text{conf}_{\text{obs}} = \phi(f(x)_y)$ ▷ query target model
- 15: **return** $\Lambda = \frac{p(\text{conf}_{\text{obs}} \mid \mathcal{N}(\mu_{\text{in}}, \sigma_{\text{in}}^2))}{p(\text{conf}_{\text{obs}} \mid \mathcal{N}(\mu_{\text{out}}, \sigma_{\text{out}}^2))}$



Traditional MIA attacks: LiRA

Method	shadow models	multiple queries	class hardness	example hardness	TPR @ 0.001% FPR			TPR @ 0.1% FPR			Balanced Accuracy		
					C-10	C-100	WT103	C-10	C-100	WT103	C-10	C-100	WT103
Yeom et al. [70]	○	○	○	○	0.0%	0.0%	0.00%	0.0%	0.0%	0.1%	59.4%	78.0%	50.0%
Shokri et al. [60]	●	○	●	○	0.0%	0.0%	–	0.3%	1.6%	–	59.6%	74.5%	–
Jayaraman et al. [25]	○	●	○	○	0.0%	0.0%	–	0.0%	0.0%	–	59.4%	76.9%	–
Song and Mittal [61]	●	○	●	○	0.0%	0.0%	–	0.1%	1.4%	–	59.5%	77.3%	–
Sablayrolles et al. [56]	●	○	●	●	0.1%	0.8%	0.01%	1.7%	7.4%	1.0%	56.3%	69.1%	65.7%
Long et al. [37]	●	○	●	●	0.0%	0.0%	–	2.2%	4.7%	–	53.5%	54.5%	–
Watson et al. [68]	●	○	●	●	0.1%	0.9%	0.02%	1.3%	5.4%	1.1%	59.1%	70.1%	65.4%
Ye et al. [69]	●	○	●	●	–	–	–	–	–	–	60.3%	76.9%	65.5%
Ours	●	●	●	●	2.2%	11.2%	0.09%	8.4%	27.6%	1.4%	63.8%	82.6%	65.6%

LiRA surpasses previous methods, not only on TPR @ low FPR, which they proposed but also balanced accuracy, which was the widely used metric previously.



Are MIAs possible on LLMs?

- Classical MIAs assume
 1. access to training data distribution to train shadow models
 2. retraining is feasible
 3. a "member" is a labelled record
- However, for pre-trained LLMs, all three assumptions fail: training data is undisclosed, training a shadow model is too expensive and a member is a text sequence
- Given these constraints, is it even possible to detect membership from LLMs?



Pythia: A Controlled Testbed for Privacy Research

- Most LLMs are privacy black-boxes, we don't know what exactly was in the training set, so we have to make informed guesses (e.g., assuming data before a certain date must be in it).
- But an informed guess can be wrong, and if members and non-members also differ in time, an attack might be detecting that **temporal gap** rather than true membership.
- To disentangle this, we need a setting where the training set is known exactly, not estimated.
- Pythia [Biderman et al., ICML'23] is a suite of open LLMs (70M → 12B) where the data, its exact training order, and 154 checkpoints per model are all public.

<https://arxiv.org/pdf/2304.01373>



Trustworthy Large Language Models



Tanmoy Chakraborty

Pythia: A Controlled Testbed for Privacy Research

	GPT-2	GPT-3	GPT-Neo	OPT	T5	BLOOM	Pythia (ours)
Public Models	●	●	●	●	●	●	●
Public Data	.	.	●		●	●	●
Known Training Order			●			●	●
Consistent Training Order				●		●	●
Number of Checkpoints	1	1	30	2	1	8	154
Smallest Model	124M	Ada	125M	125M	60M	560M	70M
Largest Model	1.5B	DaVinci	20B	175B	11B	176B	12B
Number of Models	4	4	6	9	5	5	8



The Pile: Pythia's training data

- The Pile is a 300B token corpus, used to train the Pythia suite
- Diverse domains: Wikipedia, GitHub, ArXiv, PubMed, Pile-CC, HackerNews, DM-Math.
- Comes with a built-in train/test split both from the same time period and no overlapping documents
 - Members vs non-members are explicitly known and there is no temporal gap between them.



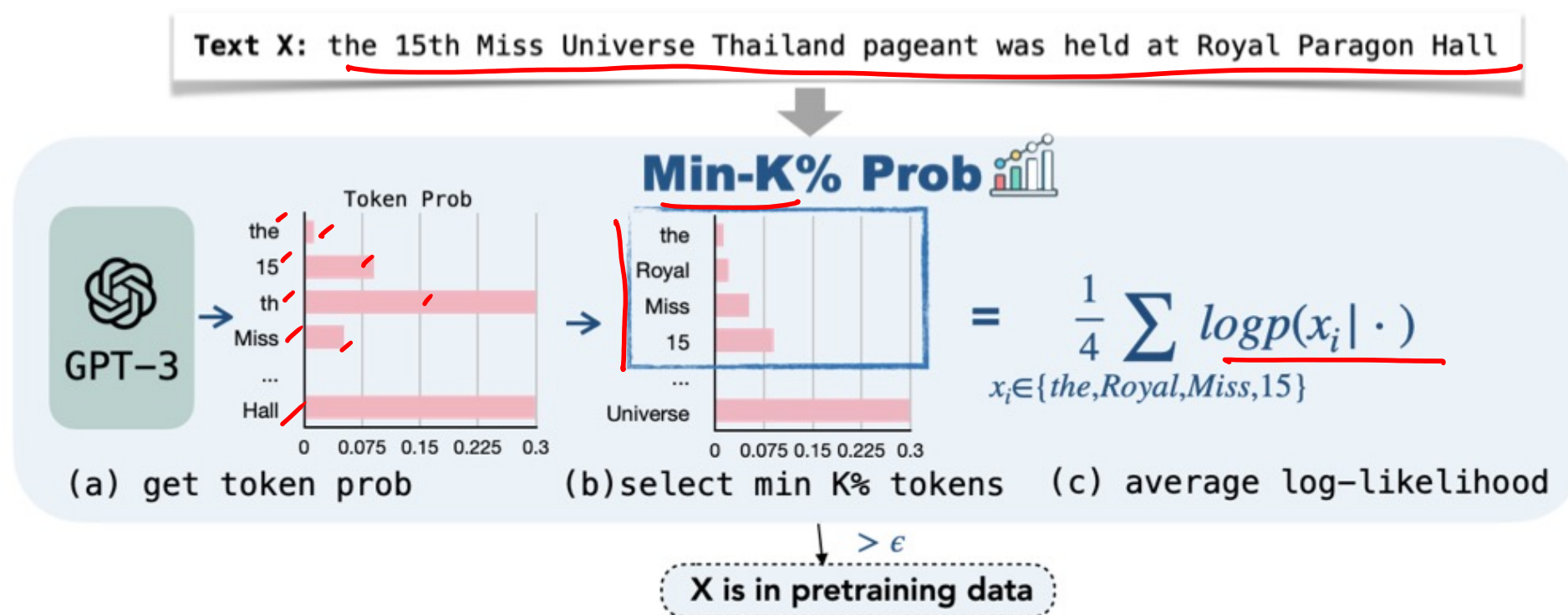
Min-k %: Detecting membership from LLM pre-training data

- **Intuition** [Shi et al. ICLR' 24]: A text sequence not present in the pre-training data is more likely to contain low probability tokens as compared to a text sequence present in the pre-training data.
- Perplexity alone is a weak indicator: even a single low-probability token can result in a high average perplexity for the whole sequence.
- Instead of focusing on the easy (high-probability) tokens, average only the k% lowest-probability tokens.

<https://arxiv.org/pdf/2310.16789>



Min-k %: Detecting membership from LLM pre-training data



Min-k %: Detecting membership from LLM pre-training data

- Consider a sequence $x = (x_1, x_2, x_3, \dots, x_n)$, where the log-likelihood of $x_i = \log p(x_i | x_1, \dots, x_{i-1})$.
- To detect membership of this sequence, select $k\%$ tokens with the least probability and form a set, $\text{Min-}k\%(x)$ and compute the average log-likelihood of tokens in this set.

$$\text{MIN-K\% PROB}(x) = \frac{1}{E} \sum_{x_i \in \text{Min-K\%}(x)} \log p(x_i | x_1, \dots, x_{i-1}). \quad E \rightarrow \text{size of the set}$$

- If this value is above a certain threshold ε , we can conclude that the sequence is a member of the pre-training dataset.
- In practice, $k = 20$, and ε isn't specifically set as the authors measure AUC

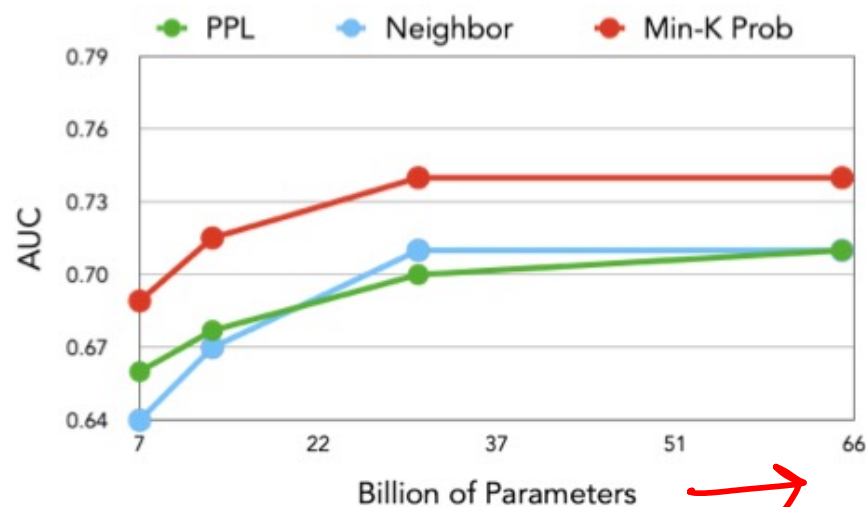


Min-k %: WikiMIA benchmark

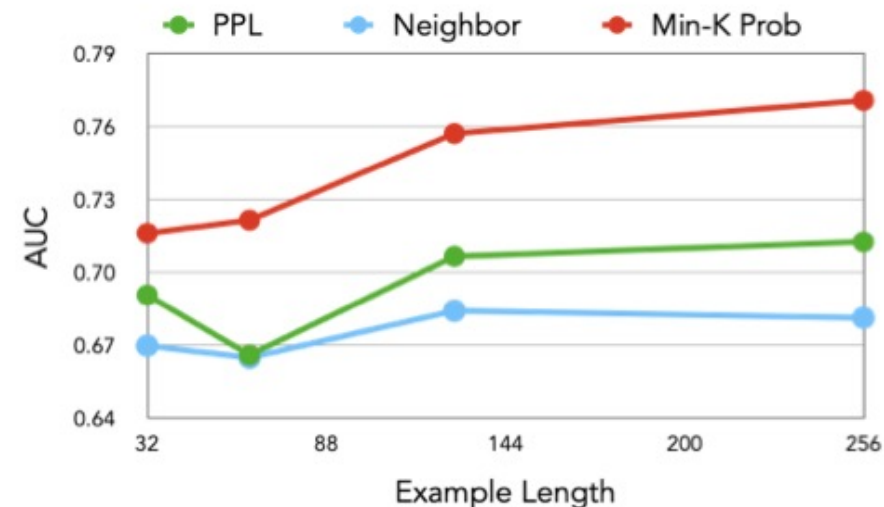
- WikiMIA benchmark: a collection of articles belonging to event category from Wikipedia.
- Non-members: Articles describing events that occurred after January 1, 2023 (the chosen cutoff date).
- Members: Articles about events that occurred before 2017, since most target models were released after 2017 and likely include Wikipedia dumps up to that period in their pre-training data.
- Target Models: LLaMA-65B, LLaMA-30B, GPT-NeoX-20B, Pythia-2.8B, and OPT-66B.
- Key Idea: The benchmark evaluates memorization beyond exact text matching by also considering paraphrased and semantically equivalent sequences, not just verbatim copies.



Min-k %: Results



(a) AUC score vs. model size



(b) AUC score vs. text length

Membership detection becomes easier with larger models and longer text sequences.



Min-k %: Results

Note: The metric used here is AUC

Method	Pythia-2.8B		NeoX-20B		LLaMA-30B		LLaMA-65B		OPT-66B		Avg.
	<i>Ori.</i>	<i>Para.</i>	<i>Ori.</i>	<i>Para.</i>	<i>Ori.</i>	<i>Para.</i>	<i>Ori.</i>	<i>Para.</i>	<i>Ori.</i>	<i>Para.</i>	
Neighbor	0.61	0.59	0.68	0.58	0.71	0.62	0.71	0.69	0.65	0.62	0.65
PPL	0.61	0.61	0.70	0.70	0.70	0.70	0.71	0.72	0.66	0.64	0.67
Zlib	0.65	0.54	0.72	0.62	0.72	0.64	0.72	0.66	0.67	0.57	0.65
Lowercase	0.59	0.60	0.68	0.67	0.59	0.54	0.63	0.60	0.59	0.58	0.61
Smaller Ref	0.60	0.58	0.68	0.65	0.72	0.64	0.74	0.70	0.67	0.64	0.66
MIN-K% PROB	0.67	0.66	0.76	0.74	0.74	0.73	0.74	0.74	0.71	0.69	0.72

Min-k% performs better on detecting original and paraphrased sequences compared to PPL as well as reference-based method



Min-k %: Results

Note: The metric used here is TPR @ 5% FPR

Method	Pythia-2.8B		NeoX-20B		LLaMA-30B		LLaMA-65B		OPT-66B		Avg.
	Ori.	Para.	Ori.	Para.	Ori.	Para.	Ori.	Para.	Ori.	Para.	
Neighbor	10.2	16.2	15.2	19.3	20.1	17.2	17.2	20.0	17.3	18.8	17.2
PPL	9.4	18.0	17.3	24.9	23.7	18.7	16.5	23.0	20.9	20.1	19.3
Zlib	18.7	18.7	20.3	22.1	18.0	20.9	23.0	23.0	21.6	20.1	20.6
Lowercase	10.8	7.2	12.9	12.2	10.1	6.5	14.4	12.2	14.4	8.6	10.9
Smaller Ref	10.1	10.1	15.8	10.1	10.8	11.5	15.8	21.6	15.8	10.1	13.2
MIN-K% PROB	13.7	15.1	21.6	27.3	22.3	25.9	20.9	30.9	21.6	23.0	22.2

Despite performing better than the baselines, the method still struggles
at TPR @ 5% FPR



Do the classical attacks transfer to LLMs?

Duan et al. [COLM '24] asked: do the classical attacks transfer to LLMs?

- **Finding:** Shadow-model based attack and LiRA attack [[Carlini et al. IEEE S&P '22](#)] give near-random AUC (≈ 0.5 - 0.6)
- Apparent successes (e.g. Min-k%) often **exploit distribution shift** between members and non-members, not real leakage

Why it is harder on LLMs?

- **Scale:** Trillions of training tokens, reducing the contribution of each record
- **Single epoch training:** less overfitting, so a weaker per-record signal
- **Ambiguous granularity:** is the sample a document, passage, or a sentence?
- **Shadow models infeasible:** training even one shadow model is really expensive

<https://arxiv.org/pdf/2402.07841>



The LLM-MIA Testbed: Setup

- Benchmark: MIMIR, built on the Pile, evaluated on Pythia suite
- Members are drawn from Pile's training split, non-members from its test split, same source and era, so membership is isolated from temporal shift.
- MIMIR ships overlap-controlled variants: higher n -gram overlap between members and non-members blurs the boundary and makes the task harder



On LLMs, output signals aren't enough

	Wikipedia					Github					Pile CC					PubMed Central				
# Params	LOSS	Ref	min-k	zlib	Ne	LOSS	Ref	min-k	zlib	Ne	LOSS	Ref	min-k	zlib	Ne	LOSS	Ref	min-k	zlib	Ne
70M	.503	.504	.494	.508	.510	.629	.584	.627	.648	.635	.494	.489	.503	.495	.489	.502	.516	.510	.502	.485
160M	.504	.515	.488	.514	.513	.638	.591	.634	.656	.638	.497	.497	.503	.498	.496	.500	.516	.504	.500	.486
1.4B	.510	.544	.506	.518	.518	.656	.587	.654	.670	.650	.500	.525	.509	.502	.499	.496	.530	.505	.500	.490
2.8B	.516	.565	.511	.522	.517	.707	.657	.708	.717	.698	.501	.537	.509	.503	.502	.498	.536	.502	.500	.497
6.9B	.514	.571	.512	.521	.514	.672	.573	.675	.684	.654	.511	.564	.516	.512	.505	.504	.552	.508	.504	.497
12B	.516	.579	.517	.524	.520	.678	.559	.683	.690	.660	.516	.582	.521	.517	.514	.506	.559	.512	.506	.497

	ArXiv					DM Math					HackerNews					The Pile				
# Params	LOSS	Ref	min-k	zlib	Ne	LOSS	Ref	min-k	zlib	Ne	LOSS	Ref	min-k	zlib	Ne	LOSS	Ref	min-k	zlib	Ne
70M	.506	.481	.499	.495	.496	.492	.520	.495	.485	.481	.494	.495	.507	.497	.506	.503	.511	.508	.506	.499
160M	.507	.486	.501	.500	.507	.490	.523	.493	.482	.489	.492	.490	.497	.497	.505	.502	.511	.506	.505	.499
1.4B	.513	.510	.511	.508	.511	.486	.512	.497	.481	.465	.503	.514	.509	.502	.504	.504	.521	.508	.507	.504
2.8B	.517	.531	.522	.512	.519	.485	.504	.497	.482	.467	.510	.549	.518	.507	.513	.507	.530	.512	.510	.506
6.9B	.521	.538	.524	.516	.519	.485	.508	.496	.481	.469	.513	.546	.528	.508	.512	.510	.549	.516	.512	.510
12B	.527	.555	.530	.521	.519	.485	.512	.495	.481	.475	.518	.565	.533	.512	.515	.513	.558	.521	.515	.511

On MIMIR, AUC clusters near 0.5-0.6 across methods and scales



MIA in open-source LLMs

After eliminating the distribution shift confound, most black-box methods give near random AUC. Can we do any better in white-box settings?

- **Look inside the model:** hidden states and attention carry traces
- **Use gradients, not outputs:** members have lower, more stable gradient norms

MIA on LLMs is hard in the black-box, output-only, pretraining setting, not impossible in general.



MIA from internal representations (*memTrace*)

Makhija et al. [EACL '26] proposed training a MIA classifier with the following:

1) LLM-head outputs - statistical aggregates across token position t

- ✓ Entropy

$$\text{entropy}[t] = - \sum_{v \in V} p_t(v) \log p_t(v)$$

where $p_t(v)$ is the softmax probability of vocabulary item v

- ✓ Prediction confidence $\text{confidence}[t]$ = $\max_{v \in V} p_t(v)$

- Confidence gap between two most probable tokens $\text{confidence_gap}[t]$ = $p_t(v_1) - p_t(v_2)$

<https://aclanthology.org/2026.eacl-long.262.pdf>



MIA from internal representations (*memTrace*)

2) Attention Heads – patterns in how the model routes information

- Attention entropy:

$$\underbrace{\frac{1}{n} \sum_{t=1}^n}_{\text{Averaged across all query rows}} \underbrace{\left(- \sum_{s=1}^n \bar{a}_{t,s}^l \log_2(\bar{a}_{t,s}^l + \epsilon) \right)}_{\text{entropy of query row } t}$$

$\bar{a}^l$ -> average of all attention heads at layer l
 t -> query position
 s -> key position
 ϵ -> small constant
 n -> sequence length

- Attention concentration: $\frac{1}{n} \sum_{t=1}^n \max_s \bar{a}_{t,s}^l$

- Attention sparsity: $\text{mean}(\bar{A}^l < \tau)$

τ -> adaptively selected threshold

- Head-focus for each head h : $\frac{1}{n} \sum_{t=1}^n \max_s a_{t,s}^{l,h}$



MIA from internal representations (*memTrace*)

3) Layer transitions – statistical aggregates for two consecutive layers l and $l+1$ across all token positions t

- Transition surprise: $\left\| \mathbf{h}_t^{(l+1)} - \mathbf{h}_t^{(l)} \right\|_2$ $\mathbf{h}_t^l \rightarrow$ hidden state at layer l at token position t
- Representation stability: $\cos(\mathbf{h}_t^{(l+1)}, \mathbf{h}_t^{(l)})$

4) Context Evolution - how a model's hidden state changes as more tokens are processed

$$\left\| \frac{1}{i+1} \sum_{t=0}^i \mathbf{h}_t - \frac{1}{i} \sum_{t=0}^{i-1} \mathbf{h}_t \right\|_2$$



MIA from internal representations (*memTrace*)

5) Activation Patterns: ✓

- Sparsity – fraction of near zero activation
- Peak activation – maximum absolute value
- Activation entropy – distribution concentration
- Neuron utilization – fraction of active neurons
- Activation Ratios – balance between +ve and –ve activations

6) Token-position specific features

- First-to-last similarity: similarity between hidden states at first and last token

$$\cos(\mathbf{h}_0, \mathbf{h}_{n-1})$$

- Confidence variation: how confidence changes in the neighbourhood of token position k

$$\sigma(\{\text{confidence}[t]\}_{t=k-c}^{k+c})$$



MIA from internal representations (*memTrace*): Results

- **Datasets:** MIMIR, WikiMIA, BookMIA
- **MIA Classifier:** Random Forest
- **Models:** Pythia Suite, LLaMa-7B, GPT-Neo (1.3B & 2.7B)

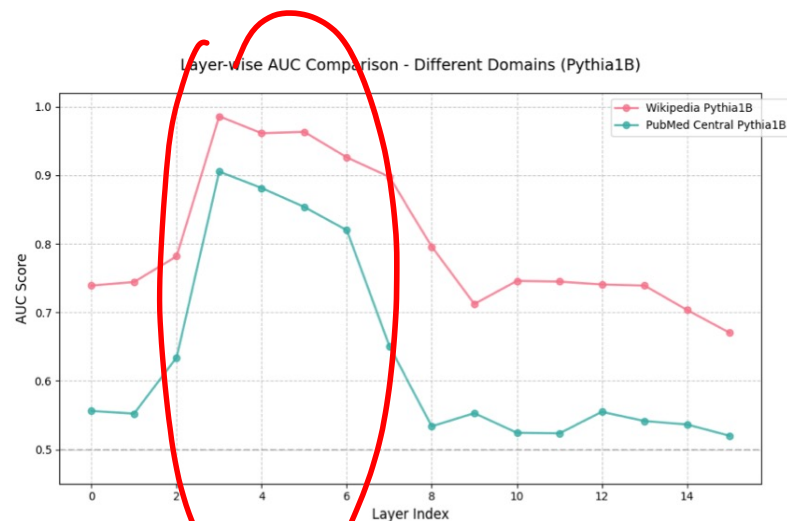
Method	Wikipedia							Pubmed Central							Hacker News						
	Pythia-70M	Pythia-410M	Pythia-1B	Pythia-6.9B	LLaMA-7B	GPTNeo-1.3B	GPTNeo-2.7B	Pythia-70M	Pythia-410M	Pythia-1B	Pythia-6.9B	LLaMA-7B	GPTNeo-1.3B	GPTNeo-2.7B	Pythia-70M	Pythia-410M	Pythia-1B	Pythia-6.9B	LLaMA-7B	GPTNeo-1.3B	GPTNeo-2.7B
Perplexity	0.51	0.50	0.50	0.52	0.54	0.51	0.51	0.50	0.49	0.50	0.52	0.51	0.49	0.50	0.49	0.49	0.50	0.52	0.51	0.51	0.51
Min-K%	0.47	0.48	0.51	0.51	0.49	0.52	0.52	0.51	0.50	0.51	0.53	0.53	0.50	0.51	0.51	0.50	0.50	0.51	0.52	0.52	0.52
Min-K++%	0.50	0.50	0.52	0.56	0.51	0.52	0.53	0.50	0.49	0.51	0.53	0.50	0.51	0.51	0.50	0.50	0.52	0.54	0.49	0.51	0.52
Lowercase	0.51	0.47	0.49	0.52	0.48	0.51	0.50	0.51	0.51	0.50	0.52	0.53	0.52	0.52	0.50	0.50	0.51	0.51	0.52	0.52	0.52
Zlib	0.52	0.51	0.52	0.53	0.54	0.52	0.52	0.51	0.50	0.50	0.52	0.52	0.51	0.51	0.50	0.50	0.50	0.53	0.53	0.50	0.51
Neighborhood	0.53	0.52	0.52	0.53	0.54	0.53	0.53	0.48	0.48	0.49	0.53	0.54	0.51	0.51	0.50	0.51	0.50	0.52	0.52	0.50	0.51
ReCaLL	0.51	0.50	0.52	0.54	0.50	0.52	0.52	0.52	0.50	0.50	0.51	0.50	0.50	0.51	0.53	0.53	0.53	0.54	0.50	0.53	0.54
MIATuner	0.49	0.51	0.51	0.49	0.50	0.50	0.49	0.50	0.50	0.48	0.50	0.50	0.50	0.52	0.50	0.50	0.53	0.50	0.50	0.49	0.49
FSD	0.48	0.47	0.49	0.47	0.50	0.50	0.49	0.49	0.49	0.49	0.47	0.48	0.48	0.49	0.49	0.49	0.49	0.47	0.50	0.49	0.47
memTrace (Ours)	0.65	0.87	0.89	0.89	0.90	0.86	0.86	0.59	0.89	0.88	0.84	0.80	0.66	0.72	0.82	0.81	0.82	0.83	0.85	0.71	0.78

Method	WikiMIA							BookMIA							Github						
	Pythia-70M	Pythia-410M	Pythia-1B	Pythia-6.9B	LLaMA-7B	GPTNeo-1.3B	GPTNeo-2.7B	Pythia-70M	Pythia-410M	Pythia-1B	Pythia-6.9B	LLaMA-7B	GPTNeo-1.3B	GPTNeo-2.7B	Pythia-70M	Pythia-410M	Pythia-1B	Pythia-6.9B	LLaMA-7B	GPTNeo-1.3B	GPTNeo-2.7B
Perplexity	0.58	0.60	0.60	0.65	0.64	0.61	0.63	0.57	0.59	0.59	0.61	0.58	0.59	0.61	0.61	0.61	0.63	0.65	0.67	0.68	0.70
Min-K%	0.57	0.59	0.60	0.67	0.63	0.62	0.61	0.59	0.61	0.62	0.64	0.62	0.62	0.64	0.62	0.62	0.64	0.69	0.67	0.66	0.69
Min-K++%	0.50	0.62	0.64	0.72	0.85	0.65	0.68	0.50	0.40	0.45	0.55	0.57	0.40	0.42	0.50	0.67	0.69	0.73	0.61	0.68	0.70
Lowercase	0.54	0.57	0.57	0.59	0.58	0.59	0.60	0.59	0.59	0.60	0.65	0.64	0.60	0.59	0.59	0.58	0.60	0.59	0.58	0.59	0.60
Zlib	0.59	0.60	0.59	0.62	0.60	0.62	0.61	0.60	0.61	0.61	0.62	0.63	0.62	0.61	0.63	0.66	0.67	0.70	0.71	0.70	0.70
Neighborhood	0.53	0.55	0.54	0.66	0.64	0.63	0.65	0.60	0.62	0.60	0.68	0.65	0.65	0.68	0.64	0.64	0.65	0.69	0.68	0.66	0.67
ReCaLL	0.76	0.87	0.86	0.89	0.67	0.82	0.84	0.70	0.85	0.84	0.87	0.69	0.81	0.83	0.62	0.67	0.69	0.73	0.63	0.70	0.72
MIATuner	0.50	0.96	0.98	0.95	0.93	0.89	0.96	0.35	0.96	0.98	0.98	0.97	0.98	0.92	0.59	0.66	0.70	0.72	0.70	0.68	0.71
FSD	0.81	0.89	0.92	0.91	0.90	0.92	0.92	0.95	0.98	0.98	0.98	0.99	0.98	0.98	0.45	0.46	0.42	0.37	0.49	0.49	0.47
memTrace (Ours)	0.82	0.87	0.80	0.87	0.94	0.79	0.85	0.91	0.94	0.95	0.95	0.98	0.98	0.99	0.67	0.70	0.73	0.77	0.78	0.72	0.72

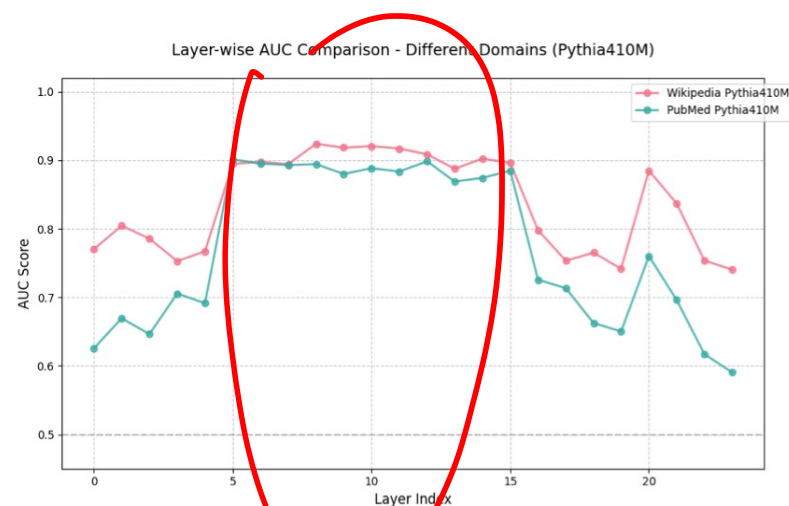
AUC is no longer near random



MIA from internal representations: Results



(a) Pythia1B



(b) Pythia410M

Features from the middle layers are better discriminators for membership



Membership signal from gradients (*OR-MIA*)

Song et al. [AAAI '26] proposed another way to detect membership in a white-box settings: through **gradient norms**

Intuition

- Gradient norms quantify the magnitude of the parameter update that would be driven by a sample. For a sample the model has seen before, it should be lower.
- Progressive and controlled perturbations to a known sample will still lie in the model's learned manifold, thus resulting in stable gradient norms.
- For non-members which lie away from the learned manifold, perturbations to the input are expected to result in volatile gradient updates.

<https://ojs.aaai.org/index.php/AAAI/article/view/40587>



Membership signal from gradients (*OR-MIA*)

The attack

- Draw $N = 10$ perturbations with growing scale $x_i = x + \xi_i$ where $\xi_i \sim N(0, \sigma^2 I)$ and take gradient norm at each:

$$g_i = \|\nabla_{\theta} \ell(f_{\theta}(x_i), y)\|_2$$

f_{θ} -> target model
 x -> input
 y -> target output

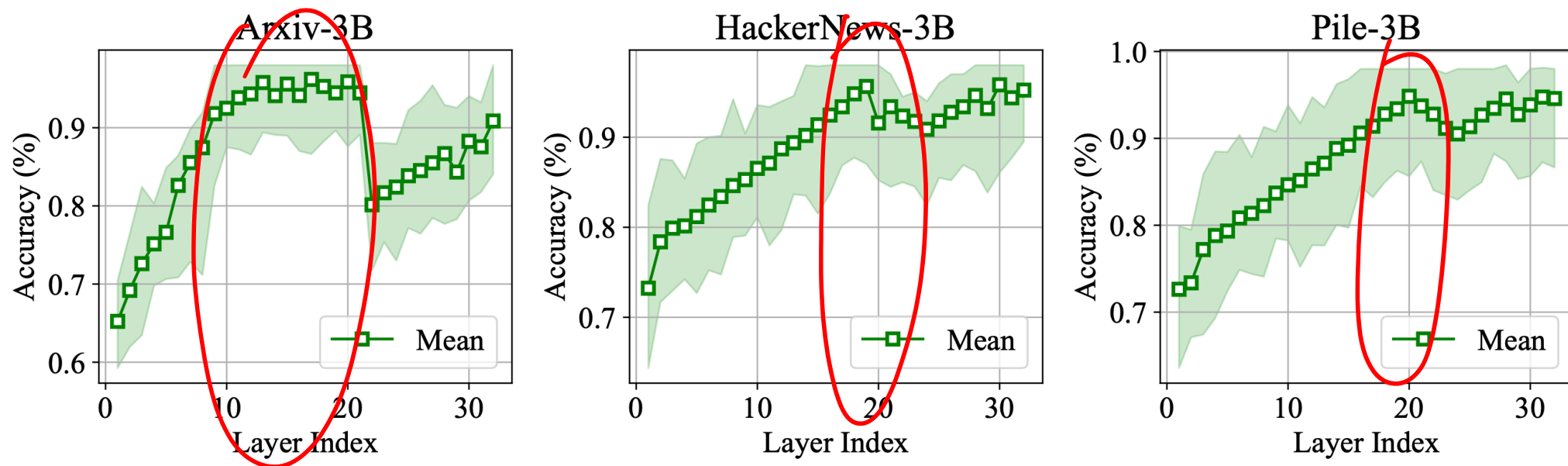
- Form a feature vector from the set of all gradient norms

$$v_x = (g_1, g_2, \dots, g_N)$$

- Classify feature vector v_x using a RBF-SVM, to be that of a member or non-member



Membership signal in gradients (*OR-MIA*): Results



Even with gradients, middle layers offer the best signals for membership inference, generally membership accuracy >90%

